

NileFlow

Setup & Run Guide

Real-Time Traffic & Transit Congestion Intelligence Platform
Greater Cairo and Alexandria

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Table of Contents

1. Prerequisites
2. Clone and Configure
3. Start the Platform
4. Verify Services
5. Run Spark Streaming Jobs
6. Access the Dashboard
7. Access Airflow
8. View Discord Alerts
9. Useful Commands
10. Troubleshooting
11. Architecture Overview
12. Database Schemas Reference

1. Prerequisites

Required Software

Software	Version	Download
Docker	20.10+	https://docs.docker.com/get-docker/
Docker Compose	v2.0+	Included with Docker Desktop
Git	2.30+	https://git-scm.com/downloads

System Requirements

- **RAM:** At least 8 GB allocated to Docker (recommended: 12 GB)
- **Disk:** At least 5 GB free space for Docker images and volumes
- **OS:** Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)

API Keys

API	Required?	Free Tier	Signup
TomTom Routing API	Yes	2,500 req/day	developer.tomtom.com
Discord Webhook	Optional	Unlimited	Discord Server Settings
Open-Meteo	No key needed	Unlimited	open-meteo.com

2. Clone and Configure

2.1 Clone the Repository

```
git clone https://github.com/mohamed-mahmoud-de/NileFlow.git
cd NileFlow
```

2.2 Create Your Environment File

```
cp .env.example .env
```

2.3 Edit the .env File

Open **.env** in any text editor and fill in your values:

```
# REQUIRED: Get your free key from developer.tomtom.com
TOMTOM_API_KEY=your_actual_api_key_here

# OPTIONAL: Discord webhook for real-time alerts
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...

# The rest can stay as defaults for local development
POSTGRES_DB=nileflow
POSTGRES_USER=nileflow
POSTGRES_PASSWORD=nileflow_dev
KAFKA_BOOTSTRAP_SERVERS=kafka:9092
REDIS_HOST=redis
CASSANDRA_KEYSPACE=nileflow
SPARK_MASTER=spark://spark-master:7077
```

3. Start the Platform

3.1 Start All Services

```
docker-compose up -d
```

This starts **18 services**. On first run, Docker pulls all images (~5 GB). Subsequent starts are much faster.

3.2 What Happens Automatically

1. Zookeeper and Kafka start (message broker)
2. PostgreSQL starts and auto-runs schema SQL + corridor seed data
3. Cassandra starts; cassandra-init waits then creates keyspace + tables
4. Redis starts (alert pub/sub + vehicle position cache)
5. Elasticsearch starts; elasticsearch-init creates alert indices
6. Airflow init migrates DB and creates admin user (admin/admin)
7. Airflow webserver and scheduler start with all Python dependencies
8. Spark master and worker start
9. 3 Kafka producers start streaming data immediately
10. Streamlit dashboard starts on port 8501
11. Discord alerter subscribes to Redis Pub/Sub

3.3 Wait for Initialization

Some services take time to initialize (especially Cassandra and Elasticsearch). Wait about **2-3 minutes** for everything to be ready.

Monitor startup progress:

```
docker-compose logs -f cassandra-init elasticsearch-init
```

4. Verify Services

Check all containers are running:

```
docker-compose ps
```

All services should show **Up** status except cassandra-init, elasticsearch-init, and airflow-init which exit after completing initialization.

Verify data flow:

```
# Check producer logs
docker logs nileflow-traffic-producer --tail 5
docker logs nileflow-weather-producer --tail 5
docker logs nileflow-vehicle-positions-producer --tail 5

# Check Cassandra has data
docker exec -it nileflow-cassandra cqlsh -e \
"SELECT COUNT(*) FROM nileflow.weather_metrics;"

# Check Elasticsearch indices
curl http://localhost:9200/_cat/indices?v
```

5. Run Spark Streaming Jobs

The Spark streaming jobs are the core processing layer. They consume from Kafka, compute metrics, detect anomalies, and write to Cassandra/Elasticsearch/Redis.

5.1 Traffic Processor (Primary)

Computes congestion index and triggers anomaly alerts

```
docker exec nileflow-spark-master /opt/spark/bin/spark-submit \  
--master spark://spark-master:7077 \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6,com.datastax.spark:spark-cassandra-connector_2.12:3.5.1 \  
/opt/nileflow/spark/streaming/traffic_processor.py
```

5.2 Weather Processor

Cleans weather data and writes to Cassandra

```
docker exec nileflow-spark-master /opt/spark/bin/spark-submit \  
--master spark://spark-master:7077 \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6,com.datastax.spark:spark-cassandra-connector_2.12:3.5.1 \  
/opt/nileflow/spark/streaming/weather_processor.py
```

5.3 Vehicle Positions Processor

Aggregates vehicle speeds per corridor

```
docker exec nileflow-spark-master /opt/spark/bin/spark-submit \  
--master spark://spark-master:7077 \  
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6,com.datastax.spark:spark-cassandra-connector_2.12:3.5.1 \  
/opt/nileflow/spark/streaming/vehicle_positions_processor.py
```

Note: On first run, Spark downloads connector JARs (1-2 minutes). Subsequent runs use cached JARs.

6. Access the Dashboard

Open your browser and go to: **http://localhost:8501**

The Streamlit dashboard shows:

- Header metrics: corridors monitored, avg congestion, avg speed, anomalies, vehicles
- Live congestion map: color-coded corridors (green=free flow, red=gridlock) with vehicle markers
- Corridor status table: real-time congestion level and speed per corridor
- Time-series charts: congestion index and speed over the last 2 hours
- Alert feed: latest anomaly alerts from Elasticsearch
- Weather sidebar: current conditions for Cairo and Alexandria

The dashboard auto-refreshes every 10 seconds.

7. Access Airflow

Open your browser: **http://localhost:8080**

Login: **admin / admin**

DAG	Schedule	Purpose
nileflow_daily_data_refresh_...	@daily	Upsert corridors + recalculate baselines
nileflow_data_quality_...	Every 30 min	Kafka/Cassandra/ES freshness checks

8. View Discord Alerts

If you configured `DISCORD_WEBHOOK_URL` in `.env`, the alerter posts formatted alerts with severity colors whenever Spark detects anomalies.

```
docker logs nileflow-discord-alerter --tail 10
```

9. Useful Commands

View Logs

```
docker-compose logs -f # All services
docker-compose logs -f traffic-producer # Specific service
```

Query Cassandra

```
docker exec -it nileflow-cassandra cqlsh
USE nileflow;
SELECT * FROM congestion_metrics LIMIT 10;
SELECT * FROM weather_metrics LIMIT 10;
```

Query PostgreSQL

```
docker exec -it nileflow-postgres psql -U nileflow -d nileflow
SET search_path TO nileflow;
SELECT * FROM corridors;
SELECT * FROM congestion_baselines LIMIT 10;
```

Query Elasticsearch

```
curl http://localhost:9200/_cluster/health?pretty
curl http://localhost:9200/congestion_alerts/_search?pretty&size=5
```

Redis

```
docker exec -it nileflow-redis redis-cli
GET nileflow:vehicle_positions
SUBSCRIBE nileflow:alerts
```

Stop and Restart

```
docker-compose down # Stop (keeps data)
docker-compose down -v # Stop + delete all data
docker-compose restart dashboard # Restart specific service
```

10. Troubleshooting

Cassandra Init Fails

Cassandra takes 60-90 seconds to start. If cassandra-init exits early:

```
docker-compose restart cassandra-init
```

Producers Keep Restarting

Check logs. Common causes: Kafka not ready (wait 30s), missing API key in .env, network errors.

```
docker logs nileflow-traffic-producer --tail 20
```

Dashboard Shows No Data

Ensure: (1) producers are running, (2) Spark processors have been submitted, (3) Cassandra has data.

```
docker exec -it nileflow-cassandra cqlsh -e \  
"SELECT COUNT(*) FROM nileflow.congestion_metrics;"
```

Airflow DAG Import Errors

Python dependencies may not have installed. Restart Airflow:

```
docker-compose restart airflow-scheduler airflow-webserver
```

Out of Memory

Increase Docker memory: Docker Desktop > Settings > Resources > Memory > 10+ GB.

Port Conflicts

Stop the conflicting service or change port mapping in docker-compose.yml.

11. Architecture Overview

End-to-End Data Flow

```
Data Sources --> Kafka Topics --> Spark Streaming --> Storage Layer
| |
| anomalies |
v v
Redis Pub/Sub Streamlit Dashboard
| (map, charts, alerts)
v
Discord Alerts

Airflow: daily refresh, baselines, data quality checks
```

All Docker Services

#	Service	Port	Purpose
1	zookeeper	2181	Kafka coordination
2	kafka	9092, 29092	Event streaming
3	postgres	5432	Reference data + Airflow metadata
4	cassandra	9042	Time-series metrics
5	redis	6379	Alert pub/sub + position cache
6	elasticsearch	9200	Alert and event log search
7	cassandra-init	-	Auto-creates keyspace + tables
8	elasticsearch-init	-	Auto-creates indices
9	airflow-init	-	DB migration + admin user
10	airflow-webserver	8080	DAG management UI
11	airflow-scheduler	-	DAG execution engine
12	spark-master	7077, 8081	Spark cluster manager
13	spark-worker	-	Spark execution (2 cores, 2 GB)
14	traffic-producer	-	TomTom API polling
15	weather-producer	-	Open-Meteo API polling
16	vehicle-positions-producer	-	Simulated GPS pings
17	dashboard	8501	Streamlit visualization
18	discord-alerter	-	Real-time alert posting

12. Database Schemas Reference

PostgreSQL - corridors

Column	Type	Description
corridor_id	VARCHAR(50) PK	Unique ID (e.g., ring_road)
name	VARCHAR(150)	Display name
city	VARCHAR(50)	Cairo or Alexandria
start_lat / start_lon	DOUBLE PRECISION	Corridor start coordinates
end_lat / end_lon	DOUBLE PRECISION	Corridor end coordinates
distance_km	DOUBLE PRECISION	Corridor length in km

PostgreSQL - congestion_baselines

Column	Type	Description
corridor_id	VARCHAR(50) PK, FK	References corridors
day_of_week	INTEGER PK	0=Monday .. 6=Sunday
hour_of_day	INTEGER PK	0..23
avg_travel_time_sec	DOUBLE PRECISION	Average travel time
sample_count	INTEGER	Number of data points
updated_at	TIMESTAMP	Last recalculation

Cassandra - congestion_metrics (30-day TTL)

Column	Type	Key	Description
corridor_id	TEXT	Partition	Corridor identifier
event_time	TIMESTAMP	Clustering DESC	Event timestamp
congestion_index	DOUBLE		0.0 (free) to 2.0+ (gridlock)
speed_kmh	DOUBLE		Average speed
is_anomaly	BOOLEAN		Anomaly flag

Cassandra - weather_metrics (30-day TTL)

Column	Type	Key	Description
location	TEXT	Partition	Cairo or Alexandria
event_time	TIMESTAMP	Clustering DESC	Observation time
temperature_c	DOUBLE		Temperature in Celsius
humidity_pct	DOUBLE		Relative humidity %
precipitation_mm	DOUBLE		Precipitation in mm
wind_speed_kmh	DOUBLE		Wind speed
weather_code	INT		WMO weather code

Elasticsearch - congestion_alerts

Field	Type	Description
corridor_id	keyword	Corridor identifier
alert_type	keyword	high_congestion or low_speed
severity	keyword	warning or critical
congestion_index	double	Congestion value at alert time
speed_kmh	double	Speed at alert time
message	text	Human-readable alert message
event_time	date	When the anomaly occurred